Proyecto 2: Animando Estructuras de Datos
CS2023 - Algoritmos y Estructuras de Datos

Luciano A. Romero Calla

2026-2

1. Descripción

En este proyecto deberán elaborar un video educativo, al estilo de **Manim**[1] (Mathematical Animation Engine), que explique y demuestre visualmente el funcionamiento de una estructura de datos implementada en C/C++. El lenguaje de programación y la herramienta de animación son de **libre elección del grupo**: no es obligatorio usar Python ni Manim específicamente. Pueden usar la combinación que prefieran (por ejemplo, Manim en Python, Motion Canvas en TypeScript, Processing, o cualquier otra librería de animación en el lenguaje que elijan), siempre que el resultado cumpla con el contenido mínimo solicitado. Se recomienda revisar los videos del canal 3Blue1Brown[2] como referencia de estilo e inspiración.

2. Estructura de datos asignada

A cada grupo se le asignará, **de forma aleatoria**, una única estructura de datos según lo efectivamente cubierto en clase al momento de asignar el proyecto (**el docente confirmará y/o ajustará esta lista** según corresponda); no se repetirá una misma estructura entre grupos de una misma sección:

- **BFS 0-1**: Variante de BFS que permite calcular los caminos mínimos desde un nodo hacia todos los demás en $O(|V| + |E|)$ en un grafo dirigido y ponderado cuyas aristas tienen peso 0 o 1.
- **Ciclo euleriano**: Ciclo que recorre todas las aristas de un grafo exactamente una vez. Existe un algoritmo lineal para encontrar un ciclo euleriano en un grafo que lo admite.
- **Topological sort (Tarjan)**: Algoritmo para obtener un ordenamiento topológico de un grafo dirigido y acíclico (DAG) en tiempo lineal, basado en DFS y en los tiempos de finalización de los nodos.
- **DSU con rollback**: Variante de DSU que permite deshacer operaciones de unión anteriores (*join*), restaurando el estado previo de la estructura. Cada operación de unión y su correspondiente reversión pueden realizarse en $O(1)$ amortizado si no se utiliza compresión de caminos.
- **Virtual tree (Auxiliary tree)**: Árbol construido a partir de un subconjunto de nodos de un árbol original, que incluye los ancestros comunes necesarios para preservar las relaciones entre dichos nodos. Se construye en $O(k \log k)$ para k nodos seleccionados, o en $O(k)$ si estos ya están ordenados por el tiempo de entrada del DFS.

[1] `https://docs.manim.community/en/stable/index.html`
[2] `https://www.3blue1brown.com/`

- **Centroide de un árbol**: Nodo cuya eliminación divide el árbol en componentes conexas de tamaño a lo más $\frac{n}{2}$. Todo árbol tiene al menos un centroide, y se puede encontrar uno en tiempo $O(n)$.
- **Centro de un árbol**: Nodo que minimiza la máxima distancia hacia cualquier otro nodo del árbol. Un árbol tiene uno o dos centros, y estos pueden encontrarse en tiempo $O(n)$, por ejemplo, eliminando iterativamente las hojas hasta que queden uno o dos nodos.
- **DFS Timestamps**: Tiempos de descubrimiento (tin) y finalización ($tout$) de los nodos durante un DFS. En árboles, permiten determinar relaciones de ancestro y verificar si un nodo pertenece al subárbol de otro en $O(1)$, una vez calculados mediante un DFS en $O(n)$.
- **Componentes biconexas**: Componentes de un grafo no dirigido que no pueden desconectarse al eliminar un único vértice. Pueden identificarse en tiempo $O(|V| + |E|)$ mediante DFS y los tiempos de descubrimiento y valores low.
- **Algoritmo de Dial**: Algoritmo de caminos mínimos para grafos con pesos enteros no negativos acotados por C. Utiliza una estructura de buckets para procesar las distancias y puede ejecutarse en $O(|E| + C|V|)$, lo que mejora la complejidad de Dijkstra cuando C es suficientemente pequeño.
- **Topological sort (Kahn)**: Algoritmo para obtener un ordenamiento topológico de un grafo dirigido y acíclico (DAG) en tiempo $O(|V| + |E|)$. Utiliza los grados de entrada de los nodos y una cola para procesar primero aquellos cuyo grado de entrada es cero.
- **Diámetro de un árbol**: Camino de mayor longitud entre dos nodos del árbol, donde la longitud se mide por el número de aristas o por la suma de sus pesos. En un árbol no ponderado, puede encontrarse en tiempo $O(n)$ mediante dos recorridos BFS o DFS.
- **Puentes**: aristas cuya eliminación aumenta el número de componentes conexas de un grafo no dirigido. Pueden encontrarse todas en tiempo $O(|V| + |E|)$ mediante DFS y los valores low.
- **DSU Tree**: Árbol construido a partir de un DSU que representa la historia de las uniones entre componentes. Permite registrar cómo se fusionan las componentes a lo largo del tiempo y puede utilizarse para responder consultas sobre el momento en que dos nodos pasan a formar parte de la misma componente. Su construcción y utilidad dependen de la variante específica de DSU Tree empleada.
- **Articulaciones**: vértices cuya eliminación aumenta el número de componentes conexas de un grafo no dirigido. Pueden encontrarse todos en tiempo $O(|V| + |E|)$ mediante DFS y los valores low.
- **SCC (Strongly Connected Components)**: Componentes fuertemente conexas de un grafo dirigido, es decir, conjuntos máximos de vértices tales que existe un camino dirigido entre cualquier par de vértices del conjunto en ambas direcciones. Pueden encontrarse todas en tiempo $O(|V| + |E|)$ mediante algoritmos como Tarjan o Kosaraju.
- **2-SAT**: Problema de satisfacibilidad de fórmulas booleanas en las que cada cláusula contiene exactamente dos literales (o, en la formulación usual, a lo más dos). Se resuelve en tiempo $O(|V|+|E|)$ construyendo el grafo de implicaciones y calculando sus componentes fuertemente conexas. La fórmula es satisfacible si y solo si ninguna variable y su negación pertenecen a la misma componente fuertemente conexa.

3. ¿Qué debe incluir el video?

Se sugiere una duración mínima de 1 a 2 minutos; en cualquier caso, **no debe exceder los 5 minutos**. Debe contener como mínimo:

- Motivación inicial: planteamiento de un problema para el cual la estructura o el algoritmo es eficiente para resolverlo.
- Una introducción conceptual: qué es la estructura o el algoritmo; si se presenta una estructura, se debe decir qué ADT representa y para qué se usa en la práctica; si se presenta un algoritmo, se debe decir para qué sirve y dar una breve explicación de cómo funciona.
- Si se presenta una estructura de datos, se debe incluir una demostración animada de sus operaciones principales; si se presenta un algoritmo, se debe incluir una demostración animada de su ejecución sobre una instancia ilustrativa.
- Al menos un caso de borde relevante (estructura vacía, un solo elemento, peor caso, etc.).
- Un breve análisis de la complejidad temporal de las operaciones mostradas o del algoritmo presentado.
- El título del proyecto y los nombres de los integrantes al inicio del video o en los créditos finales.

Nota (la animación debe ser real, no simulada): la animación debe estar impulsada por una implementación real de la estructura, programada por el propio grupo —no se permite animar valores o pasos "a mano" sin lógica real detrás, ni usar una implementación ya existente de la estructura asignada provista por alguna librería estándar o de terceros (por ejemplo `heapq`, `std::stack`, `java.util.Stack`, si esa es justamente la estructura asignada).

4. Entrega

La entrega es grupal en Canvas y se califica con una rúbrica publicada que evalúa la implementación, la animación, la presentación, el contenido explicativo y el informe.

- Código fuente (en el lenguaje elegido) que genera la animación, alojado en un **repositorio de GitHub** con un archivo `README.md` que presente el proyecto, el software requerido y los pasos para compilar/ejecutar el código. Debe mostrar un historial convincente de progreso del trabajo; un solo commit realizado el mismo día de la entrega o una secuencia de commits que no muestre el desarrollo final del proyecto invalida su trabajo.
- Video renderizado en formato MPEG-4 (`.mp4`) y en alta resolución; incluyan un enlace (YouTube no listado, Drive, etc.) dentro del informe.
- Un informe breve en PDF: descripción de la estructura y su ADT o algoritmo, herramientas usadas, pasos para reproducir el video, contribuciones de cada integrante, análisis de complejidad, evidencia (capturas o enlace) del repositorio de GitHub y del video, y conclusiones. Asegúrense de que los enlaces funcionen correctamente. El PDF no debe exceder 4 páginas.

5. Modalidad y fecha límite

Trabajan en grupos de **hasta 3 integrantes**. La presentación en clase y la fecha límite para subir su trabajo son el **20 de noviembre de 2026**. Si se inspiran en un video o ejemplo de terceros, cítenlo explícitamente en el informe.

Registra tu grupo aquí: `https://forms.gle/KYeBQioBwPAYLHdSA`. Fecha límite: **viernes 30 de octubre**. Asignación de temas, **sábado 1 de noviembre**. Presentación de avances **miércoles 11 de noviembre**, requisito para la presentación final.